Crypto Analytics Platform — Technical Test

User Story

Lena checks the markets over coffee and hates the shouting—ten tabs, conflicting signals, no clarity. She wants one calm place where she can pick a pair (say, bitcoin-usd), slide a date range, and see price and volume breathe together with a few honest metrics on top. When a pair matters, she hits “Track,” and from then on the data arrives every five minutes, quietly and reliably. Later, she signs in and her tracked pairs follow her to any device. If the team grows, she can flip a switch to run it in the cloud—jobs on schedule, secrets safe—so everyone gets the same clear picture without the noise.

Objective

Build a crypto analytics platform from scratch in staged iterations. You will design the FE, BE, DB, and background processes, and optionally deploy to GCP.

Tech Stack (required unless marked optional)

- Frontend: React, Vite, Tailwind, TypeScript, Highcharts (Framer Motion optional)
- Backend: Python, Poetry, FastAPI, Uvicorn
- Background processes: Python + Poetry
- Database: MongoDB
- Auth: Firebase Authentication (Google + Email/Password)
- Infra (optional iteration): Google Cloud (Cloud Run, Cloud Scheduler/Jobs/Functions, Secret Manager), MongoDB Atlas

External Provider

- Use CoinGecko API for market data (`https://www.coingecko.com/en/api/documentation`).
- Pairs mean coin vs currency, where currency can be fiat or crypto (e.g., `bitcoin-usd`, `ethereum-btc`).

- Helpful endpoints:
    - `/api/v3/simple/price?ids={coinId}&vs_currencies={vs}` (latest)
    - `/api/v3/coins/{id}/market_chart?vs_currency={vs}&days={n}` (historical by days)
    - `/api/v3/coins/{id}/market_chart/range?vs_currency={vs}&from={unix}&to={unix}` (historical by range)
- Free tier rate limits apply; implement basic retry/backoff responsibly.

Iterations and Deliverables

Iteration 1 — Frontend Only with Local Mocks (required)

Goal: Ship UX with charts and flows using deterministic local mock data; no backend, no auth.

Deliverables:

- React/Vite/Tailwind app scaffold with routes and basic layout
- Components: pair selector, date range picker, metrics cards
- Highcharts: price (line) and volume (area) from FE-local mock fixtures
- Local mock module (e.g., `src/mocks/analytics.ts`) returning series for `{coinId, vsCurrency}` and date range
- Clean state management and simulated loading/empty/error states
- Branch: `feat/fe-mocks`

Constraints:

- Mocks must be FE-local only (no server mocks). Remove/replace in Iteration 2.5.

Iteration 2 — Backend + Background Processes + Scripts (required)

Goal: Build data platform services; FE not yet connected.

Deliverables:

- FastAPI project with OpenAPI docs and health endpoint
- Core endpoints (recommendations, not strict):
    - `POST /api/pairs { coinId, vsCurrency }` — start tracking
    - `GET /api/pairs`
    - `DELETE /api/pairs/{coinId}/{vsCurrency}`
    - `GET /api/analytics/{coinId}/{vsCurrency}` with `from`, `to`, `interval`
    - `GET /api/analytics/{coinId}/{vsCurrency}/latest`
- MongoDB collections with appropriate indexes
- Background collector(s) fetching every 5 minutes per active pair
- Scripts:
    - Backfill historical data for a pair and window
    - List/seed tracked pairs for local testing
- README section with commands to run API, workers, and scripts locally
- Branches:
    - `feat/be-core-api`
    - `feat/be-bg-processes`
    - `feat/dev-scripts`

Notes:

- Implement idempotent upserts keyed by `{coinId, vsCurrency, timestamp}`
- Handle rate limits and transient failures (retry/backoff)

Iteration 2.5 — Integrate FE + Firebase Auth + Replace Mocks (required)

Goal: Secure endpoints and wire FE to BE; retire FE mocks.

Deliverables:

- Firebase Auth on FE (Google + Email/Password)
- Protected routes; sign-in/sign-out UI
- BE JWT verification middleware for Firebase tokens (protect non-health endpoints)
- FE replaces mock calls with real API (remove mock module from runtime)
- Pair management UI triggers start/stop tracking via API
- Polling for latest metrics (no WebSockets/SSE)
- Branches:
    - `feat/fe-auth`
    - `feat/fe-integration`
    - `feat/be-auth-middleware`

Iteration 3 — GCP Deployment (optional)

- Backend/API on Cloud Run
- Background collectors via Cloud Run Jobs/Functions + Cloud Scheduler (5-min cadence)
- Secrets via Secret Manager; MongoDB Atlas
- Deployment scripts/manifests and documentation

- Branch: `feat/infra-gcp`

Database (MongoDB) — Suggested Collections

- `users`: `{ _id, firebaseUid, createdAt }`
- `tracked_pairs`: `{ _id, userId, coinId, vsCurrency, status: 'active'|'stopped', createdAt, updatedAt }` with unique index `(userId, coinId, vsCurrency)`
- `price_data`: `{ _id, coinId, vsCurrency, timestamp, price, volume, provider: 'coingecko' }` with indexes on `(coinId, vsCurrency, timestamp)`

Git Workflow and Branching (Recommendations)

- Create focused branches per concern (see iterations). One concern per PR; small, reviewable diffs.
- Suggested branches:
  - `feat/fe-mocks`
  - `feat/be-core-api`
  - `feat/be-bg-processes`
  - `feat/dev-scripts`
  - `feat/fe-auth`
  - `feat/fe-integration`
  - `feat/infra-gcp`

- No direct commits to `main`.

Constraints and Rules

- Keep solutions simple; avoid duplication
- Use only required stacks unless justified; document any additions
- FE mocks allowed only in Iteration 1; remove in 2.5
- Do not commit secrets; use env vars. Provide `.env.example` if you choose
- Prefer files ≤300 LOC; refactor if needed

Setup Expectations (document in your repo)

- Local run commands for FE, BE, and background workers
- Environment variables needed (document names and purpose)
- Instructions to create Firebase project and obtain client config
- Instructions to provision MongoDB (local or Atlas) and connection string

Submission

- Public Git repository with clear commit history and a top-level README
- Iteration-staged instructions and deliverables clearly documented
- Optional: short demo video or screenshots

Evaluation

1. Architecture and Clarity

- Clear separation of concerns (FE, BE, background, DB)
- Thoughtful data modeling and responsibility boundaries
- Iteration deliverables cleanly scoped and documented

2. Code Quality and Readability

- Idiomatic React/TypeScript and Python
- Naming, structure, and small, focused modules (≤300 LOC preference)
- Linting/formatting configured and followed

3. Correctness and Robustness

- 5-minute collection granularity implemented or justified alternative
- Handles rate limits, retries/backoff, and idempotent upserts
- Validations and clear error responses

4. Security

- Firebase token verification on protected BE routes (Iteration 2.5)
- Secret handling via env vars; no secrets committed
- Least privilege and safe defaults

5. UX and Product Polish

- Clean Tailwind layout and accessible components
- Intuitive flows (pair selection, date range, metrics)
- Useful empty/loading/error states

6. API Design and Documentation

- OpenAPI docs available; endpoints cohesive and consistent
- Endpoints follow recommendations or documented rationale for changes
- Clear run commands and setup instructions

7. Observability

- Structured logging; helpful context for jobs/requests
- Basic metrics or status endpoints (health, job status)
- Diagnostics for background processes

8. Deployment and Ops

- Cloud Run + Scheduler/Jobs/Functions defined if attempted
- Clear infra docs and reproducible steps
- Sensible env/secret management (e.g., Secret Manager)